

2024 Sep Oppe 1 - Set 1

Section 1 - Problem 3

Remove elements at given two indices of a list

PROBLEM STATEMENT

- Write a function `remove_elements_at_two_indices` that takes a list `l` and two indices `i1` and `i2`. The function should remove the elements at these two indices from the list `l`.
- Assume the indices are non-negative and unique.

|  In-Place Only (Modify existing object). The function should not return anything.

|  NOTE: This is a function type question, you don't have to take input or print the output, just have to complete the required function definition.

EXAMPLE & EXPLANATION

```
>>> l = [1, 2, 3, 4, 5, 6, 7]
>>> i1 = 5
>>> i2 = 1
>>> remove_elements_at_two_indices(l, i1, i2)
>>> l
[1, 3, 4, 5, 7]
```

The elements at indices **5** and **1** are **6** and **2**, respectively. These elements are removed from the list `l`.

The Challenge

INPUTS Source Sans Pro

$l = [\overset{0}{1} \quad \overset{1}{2} \quad \overset{2}{3} \quad \overset{3}{4} \quad \overset{4}{5} \quad \overset{5}{6} \quad \overset{6}{7}]$

$i1 = 5$
 $i2 = 1$

CONSTRAINTS Source Sans Pro

- |  In-Place Only (Modify existing object).
- |  No Return Value.

TARGET STATE Source Sans Pro

START ARRAY

$[\overset{0}{1} \quad \overset{1}{2} \quad \overset{2}{3} \quad \overset{3}{4} \quad \overset{4}{5} \quad \overset{5}{6} \quad \overset{6}{7}]$

TARGET ARRAY

$[\overset{0}{1} \quad \overset{1}{3} \quad \overset{2}{4} \quad \overset{3}{5} \quad \overset{4}{7}]$

The Intuitive Approach

Logic suggests we simply remove the items at the requested indices one by one.

```
# Naive Attempt
```

```
l.pop(1) # Remove first requested index
```

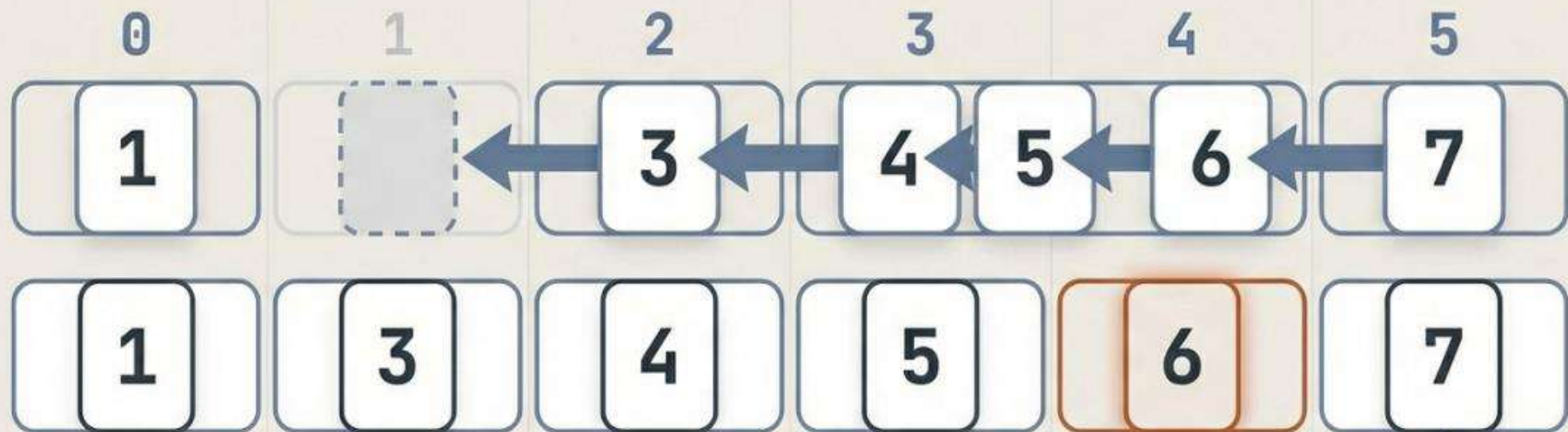
```
l.pop(5) # Remove second requested index ?
```

This creates a
hidden bug.

The Shifting Index Trap

ACTION

Execute `l.pop(1)` (Value: 2)



Value `6` has moved
from Index 5 to Index 4.

The Consequence

Action:

Execute `l.pop(5)` (Original Request)



Result: Removed `7`. Value `6` remains.

UNINTENDED RESULT

Value we wanted (which was 6) is sitting safely at Index `4`.

The Strategy Shift

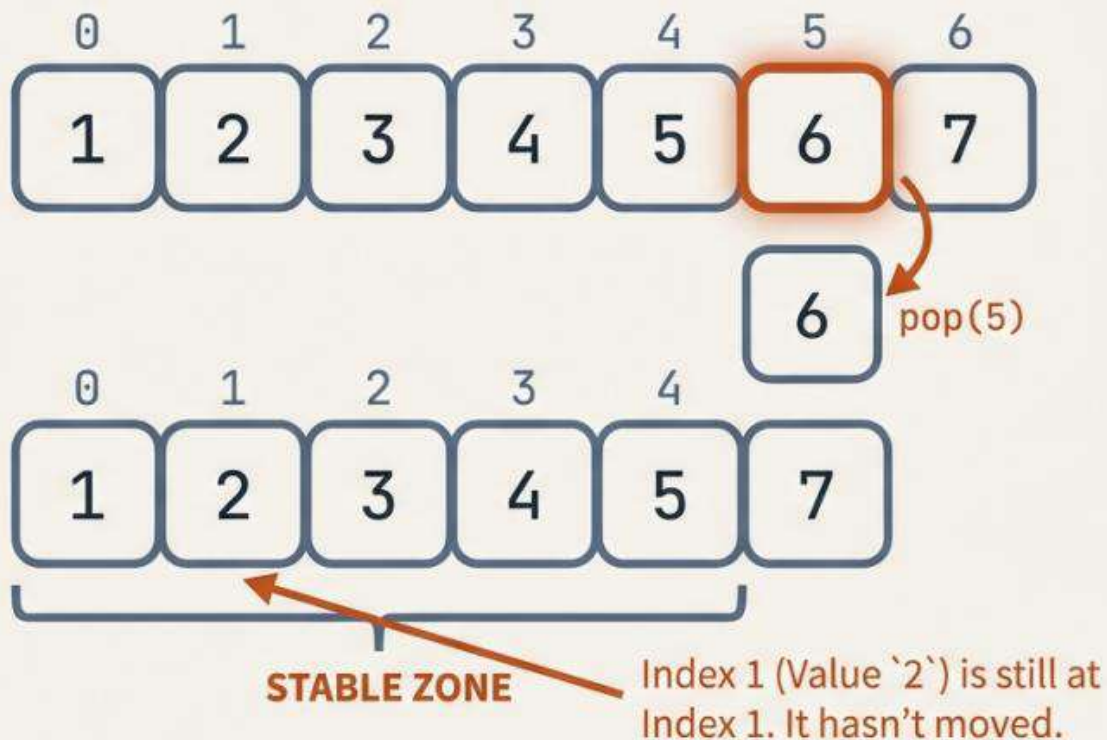
“To keep the ground from moving under your feet, start from the end.”



By removing the largest index first, the indices of all preceding elements remain unchanged.

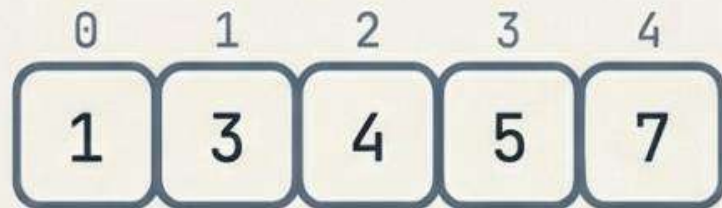
Visualizing the Fix: Step 1

Action header: Sort Indices: Remove Largest First (`i1=5`)



Visualizing the Fix: Step 2

Action header: Remove Remaining Index (`i2=1`)



Matches Target.

Lower indices are immune to changes happening at higher indices.

The Manual Solution

Explicit logic for exactly two inputs.

```
def remove_elements_at_two_indices(l, i1, i2):  
    first = max(i1, i2) # Identify larger  
    second = min(i1, i2) # Identify smaller  
  
    l.pop(first) # Safe removal (High index)  
    l.pop(second) # Safe removal (Low index)
```

Force Order:

Ensure we always process right-to-left.

Scaling Up

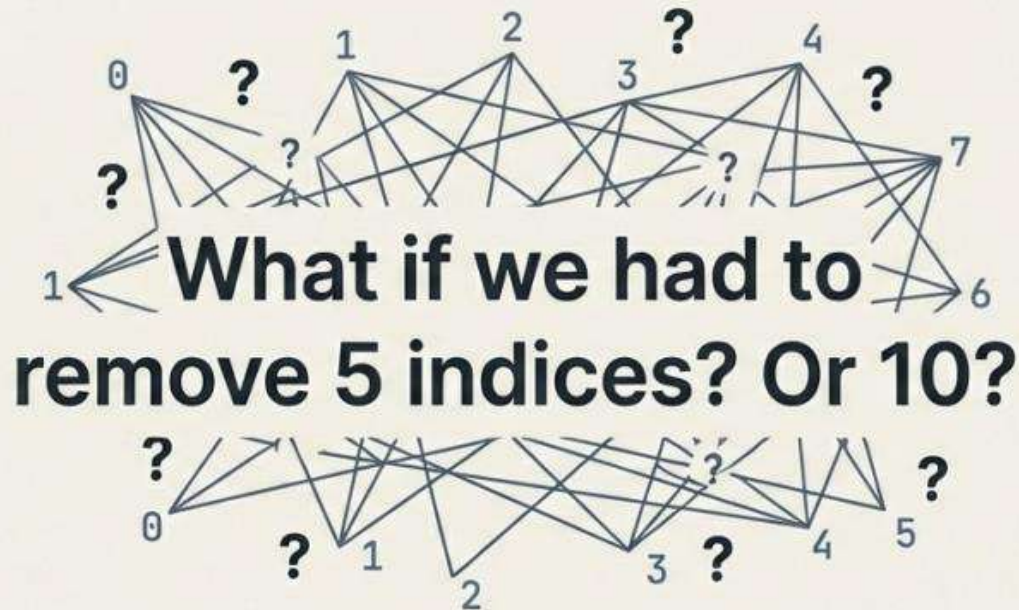
Force Order



Identify larger



Identify smaller
(max/min function)



```
l.pop(max(i1, i2, i3, i4...)) # Unmanageable logic
```

Manual comparison doesn't scale. We need a loop.

The Robust Solution

Scalable. Pythonic. Universal.

```
def remove_elements_at_two_indices(l, i1, i2):  
    # Sort indices descending to avoid shifting  
    for i in sorted([i1, i2], reverse=True):  
        l.pop(i)
```


Code Anatomy

```
for i in sorted([i1, i2], reverse=True):  
    l.pop(i)
```

Input List. Can be any length (2, 10, or 100 indices).

The Critical Flag. Forces processing from End -> Start.

In-Place Modification. No return needed.

Approach Comparison

Approach	In-Place?	Scalable?	Notes
Manual `max/min` JetBrains Mono	Yes	No	Explicit but brittle.
Sorted Loop JetBrains Mono	Yes	Yes	Standard Python practice.
List Comprehension	No	Yes	Creates a new copy (not in-place).
`del` Statement JetBrains Mono	Yes	Yes	Valid alternative syntax to `pop`.

Best Practices for List Mutation

01



Start Simple

Remember that removing elements changes the address of everything to the right.

02



Optimize Smartly

Descending Order (Reverse) is the universal key to safe index-based removal.

03



Use Native Features

Leverage `sorted()` and `pop()` rather than writing complex manual index math.

Reference Guide

THE SCALABLE POP SOLUTION

```
# Recommended
for i in sorted(indices, reverse=True):
    l.pop(i)
```

THE DEL ALTERNATIVE

```
# Alternative Syntax
for i in sorted(indices, reverse=True):
    del l[i]
```

Always `reverse=True` when removing by index.